

DJANGO CRUD APPLICATION

Creating the Model (TODO item)

In the models.py file of the todo app, define the model for a TODO item. The model will have fields like title, description, and completed (to track if the task is done).

```
from django.db import models
```

```
class Todo(models.Model):  
    title = models.CharField(max_length=255)  
    description = models.TextField()  
    completed = models.BooleanField(default=False)  
  
    def __str__(self):  
        return self.title
```

Forms for Handling Data

To handle form data, Django provides forms.ModelForm which allows for easy creation of forms based on models.

In forms.py in the todo app:

```
from django import forms  
from .models import Todo  
  
class TodoForm(forms.ModelForm):
```

```
class Meta:

    model = Todo

    fields = ['title', 'description', 'completed']
```

Creating Views for CRUD Operations

In Django, views handle the logic of interacting with data and templates. We'll define views for the four CRUD operations.

a. Create (Adding a new TODO item)

To create a new TODO item, we'll use a form where users can enter the task details.

In views.py, add the following view:

```
from django.shortcuts import render, redirect
from .models import Todo
from .forms import TodoForm

def add_todo(request):
    if request.method == 'POST':
        form = TodoForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('home')
    else:
        form = TodoForm()
    return render(request, 'add_todo.html', {'form': form})
```

b. Read (Displaying all TODO items)

The home view will fetch all the TODO items from the database and display them.

```
def home(request):  
    todos = Todo.objects.all()  
    return render(request, 'home.html', {'todos': todos})
```

Update (Editing a TODO item)

To update a TODO item, we need a form pre-populated with the existing data.

```
def update_todo(request, todo_id):  
    todo = Todo.objects.get(id=todo_id)  
    if request.method == 'POST':  
        form = TodoForm(request.POST, instance=todo)  
        if form.is_valid():  
            form.save()  
            return redirect('home')  
    else:  
        form = TodoForm(instance=todo)  
    return render(request, 'update_todo.html', {'form': form})
```

d. Delete (Removing a TODO item)

For deleting a TODO item, we just need to find the item and delete it from the database.

```
def delete_todo(request, todo_id):  
    todo = Todo.objects.get(id=todo_id)
```

```
if request.method == 'POST':  
    todo.delete()  
    return redirect('home')  
return render(request, 'delete_todo.html', {'todo': todo})
```

Setting Up URLs

Create a file called `urls.py` inside the `todo` app, if it doesn't already exist, and define the URL routes.

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.home, name='home'),  
    path('add/', views.add_todo, name='add_todo'),  
    path('update/<int:todo_id>/', views.update_todo, name='update_todo'),  
    path('delete/<int:todo_id>/', views.delete_todo, name='delete_todo'),  
]
```

Creating Templates for CRUD Operations

You will need to create templates for each view: `home.html`, `add_todo.html`, `update_todo.html`, and `delete_todo.html`. These templates should be placed inside the `todo/templates` folder.

a. `home.html` – To display all TODO items

```
<h1>TODO List</h1>
```

```
<a href="{% url 'add_todo' %}">Add a new task</a>

<ul>

  {% for todo in todos %}

    <li>

      {{ todo.title }} - {% if todo.completed %}Done{% else %}Pending{% endif %}

      <a href="{% url 'update_todo' todo.id %}">Edit</a>

      <a href="{% url 'delete_todo' todo.id %}">Delete</a>

    </li>

  {% endfor %}

</ul>
```

b. add_todo.html – For adding a new TODO item

```
<h1>Add New TODO</h1>

<form method="post">

  {% csrf_token %}

  {{ form.as_p }}

  <button type="submit">Save</button>

</form>
```

c. update_todo.html – For updating an existing TODO item

```
<h1>Edit TODO</h1>

<form method="post">

  {% csrf_token %}
```

```
{{ form.as_p }}  
<button type="submit">Update</button>  
</form>
```

d. delete_todo.html – For deleting a TODO item

```
<h1>Delete TODO</h1>  
<p>Are you sure you want to delete "{{ todo.title }}"?</p>  
<form method="post">  
    {% csrf_token %}  
    <button type="submit">Yes, Delete</button>  
</form>  
<a href="{% url 'home' %}">Cancel</a>
```